

AIT Practice Platform

Симулятор экзамена и платформа для практики программирования с AI-автогенерацией заданий

Проектная документация | Web-разработка | 2025

3

Ключевых модуля

6

Технологий стека

AI

Автогенерация тестов

MVP

Фокус разработки

Содержание

1. О проекте и решаемая проблема
2. Целевая аудитория
3. Функциональные возможности
4. AI-модуль автогенерации заданий
5. Технологический стек
6. Архитектура системы
7. Модели данных (Entity-Relationship)
8. API-эндпоинты
9. Roadmap разработки (MVP)
10. Почему этот проект важен

1. О проекте и решаемая проблема

AIT Practice Platform — это интерактивная веб-платформа, предназначенная для подготовки студентов к техническим экзаменам и отборочным этапам в IT-компаниях. Платформа симулирует реальные условия разработки, разбивая крупные проекты на пошаговые микро-задачи с моментальной автоматической проверкой кода.

Проблема

Студенты, проходящие отбор в такие компании как AIT Solution, сталкиваются с необходимостью писать архитектурный код (Model, View, Controller, бизнес-логику) вручную — на листке бумаги или в экзаменационных условиях без IDE и без права на ошибку. Это вызывает стресс, растерянность и низкие результаты даже у технически сильных студентов.

Решение

Платформа предоставляет безопасную тренировочную среду, где студент может проходить полноценные проекты по шагам, получать мгновенный фидбек от автогрейдера, тренироваться под таймером в "Exam Mode" и получать умные подсказки от AI-ассистента — не раскрывающего ответ, но направляющего к решению.

2. Целевая аудитория

Студенты 1-3 курса	Хотят освоить написание логики и архитектуры без привязки к сложной настройке окружения
Участники отборов	Готовятся к техническим экзаменам в IT-компаниях (AIT Solution и аналоги)
Преподаватели	Могут добавлять новые проекты и задания через панель администратора

3. Функциональные возможности

- Пошаговое прохождение проектов
Большой проект (например, "Система бронирования авиабилетов") разбивается на малые шаги: написать модель User, метод поиска рейсов, логику оплаты и т.д. Прогресс сохраняется.

- Браузерная IDE (Monaco Editor)
Встроенный редактор кода — тот же движок, что и в VS Code. Подсветка синтаксиса, автодополнение и удобный интерфейс прямо в браузере без установки дополнительного ПО.
- Автогрейдер (мгновенная проверка)
Студент нажимает "Run" — код выполняется изолированно на сервере, прогоняется через заранее написанные тесты. Система возвращает статус Passed / Failed с деталями.
- Exam Mode (режим экзамена)
Запуск таймера для имитации реальных условий отбора. Студент видит обратный отсчёт, не может подсматривать подсказки, и сдаёт задание строго в срок.
- Геймификация и прогресс
Шкалы прогресса по каждому проекту (например, "Airline System — 35% completed"), статусы задач (□ выполнено / □ провалено / □ в процессе), лидерборды.
- AI-Ассистент "Why failed?"
При неудаче студент может спросить AI-ассистента. Тот анализирует ошибку и даёт направляющую подсказку — не раскрывая ответ, но помогая найти правильный путь.
- Панель администратора
Преподаватель или администратор может создать новый проект, добавить уровни сложности и задать описание — AI автоматически сгенерирует задания, тесты и структуру экзамена.

4. AI-модуль автогенерации заданий

Ключевая инновационная функция платформы — автоматическое создание учебных проектов с заданиями, тестами и уровнями сложности через большую языковую модель (Claude API).

1

Администратор вводит описание проекта (название, тема, требования)

2

Описание отправляется в Claude API с системным промптом

3

AI генерирует структуру: задания, тесты, уровни (Easy/Medium/Hard)

4

Результат валидируется и сохраняется в БД



Проект становится доступен студентам на платформе

Уровни сложности заданий:

Уровень	Характеристики	Пример задания
Easy	Без таймера, доступны подсказки AI, нет ограничений по попыткам	Написать модель User с полями id, name, email
Medium	Таймер 30 мин, ограниченные подсказки (3 запроса), до 5 попыток	Реализовать метод поиска рейсов по маршруту и дате
Hard	Таймер 15 мин, подсказки недоступны, 1 попытка — Exam Mode	Написать полный контроллер бронирования с проверкой мест

5. Технологический стек

Backend

Python 3.11+	Основной язык серверной части
FastAPI	Асинхронный REST API фреймворк, автодокументация Swagger
PostgreSQL	Реляционная БД для хранения проектов, заданий, пользователей
SQLAlchemy	ORM для работы с БД, миграции через Alembic
Python exec()	MVP-инструмент для безопасного запуска кода студентов

Frontend

React + TypeScript	SPA-интерфейс с типизацией, без перезагрузок страниц
Bootstrap 5	Готовые UI-компоненты, адаптивная вёрстка
Monaco Editor	Редактор кода из VS Code, встроенный в браузер
Axios	HTTP-клиент для запросов к API

AI & Интеграции

Claude API (Anthropic)	Генерация заданий, тестов, уровней сложности, AI-ассистент
------------------------	--

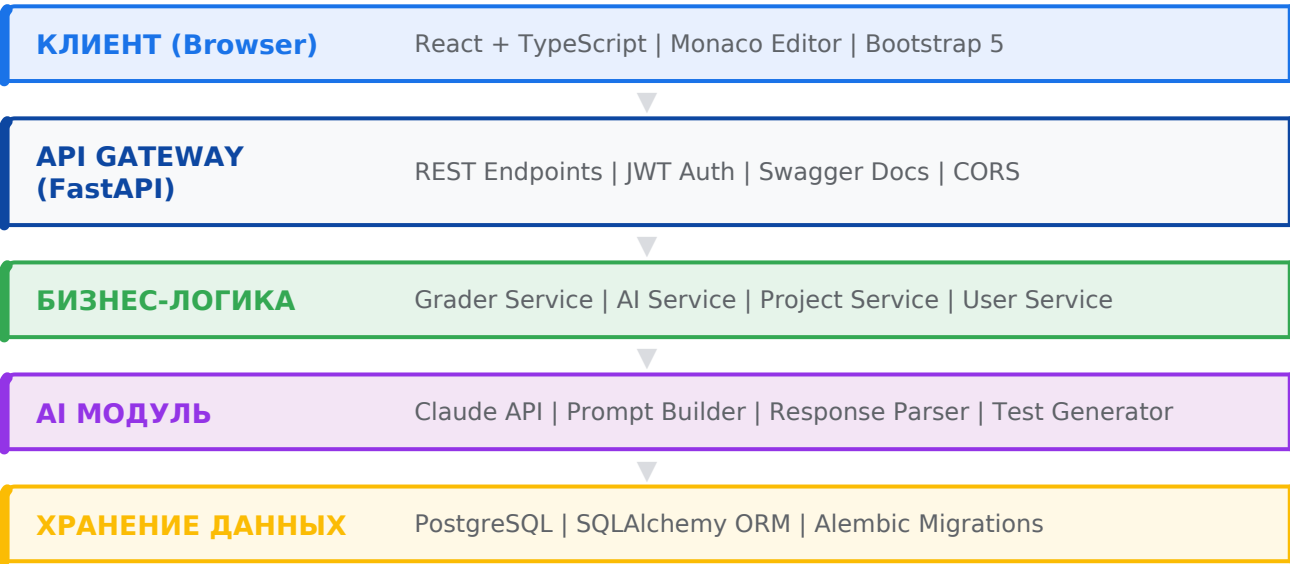
JWT	Аутентификация и авторизация пользователей
-----	--

DevOps

Git / GitHub	Контроль версий, командная разработка
Docker (v2)	Изолированный запуск кода студентов в контейнерах
Uvicorn	ASGI-сервер для FastAPI

6. Архитектура системы

Система построена по трёхуровневой архитектуре: клиентская часть (React SPA), серверная часть (FastAPI) и уровень хранения данных (PostgreSQL). AI-модуль взаимодействует с внешним Claude API для генерации контента.



7. Модели данных (основные сущности)

User			Project		
Поле	Тип	Описание	Поле	Тип	Описание
id	UUID	Первичный ключ	id	UUID	Первичный ключ
username	VARCHAR(50)	Уникальное имя	title	VARCHAR(200)	Название проекта
email	VARCHAR(100)	Email адрес	description	TEXT	Полное описание
role	ENUM	student / admin	ai_generated	BOOLEAN	Создан AI или вручную
created_at	TIMESTAMP	Дата регистрации	created_by	UUID (FK)	Ссылка на User

Task			Submission		
Поле	Тип	Описание	Поле	Тип	Описание
id	UUID	Первичный ключ	id	UUID	Первичный ключ
project_id	UUID (FK)	Ссылка на Project	task_id	UUID (FK)	Ссылка на Task
title	VARCHAR(200)	Заголовок задачи	user_id	UUID (FK)	Ссылка на User
difficulty	ENUM	easy / medium / hard	code	TEXT	Отправленный код
test_code	TEXT	Код автотестов	status	ENUM	passed / failed / pending
solution_template	TEXT	Шаблон для студента	submitted_at	TIMESTAMP	Время отправки

8. Основные API-эндпоинты

Метод	Endpoint	Описание
POST	/api/auth/register	Регистрация нового пользователя
POST	/api/auth/login	Вход, получение JWT-токена
GET	/api/projects	Список всех проектов
POST	/api/projects	Создать проект (admin)
POST	/api/projects/generate	AI-генерация проекта по описанию
GET	/api/projects/{id}/tasks	Задания конкретного проекта
POST	/api/submissions	Отправить решение на проверку
GET	/api/submissions/{id}	Статус проверки решения
POST	/api/ai/hint	Получить подсказку AI-ассистента

9. Roadmap разработки (MVP)

Фаза 1	Неделя 1-2
✓ Инициализация проекта (FastAPI + React + PostgreSQL)	
✓ Модели данных и миграции (Alembic)	

- ✓ Регистрация и авторизация (JWT)
- ✓ Базовый CRUD для проектов и заданий

Фаза 2

Неделя 3-4

- ✓ Интеграция Monaco Editor во фронтенд
- ✓ Сервис автогрейдера (exes()) для MVP
- ✓ Отправка и проверка решений
- ✓ Страница прогресса студента

Фаза 3

Неделя 5-6

- ✓ Интеграция Claude API для генерации заданий
- ✓ Панель администратора (новый проект + AI-генерация)
- ✓ AI-ассистент "Why failed?" с подсказками
- ✓ Уровни сложности (Easy / Medium / Hard)

Фаза 4

Неделя 7-8

- ✓ Exam Mode с таймером
- ✓ Геймификация: прогресс-бары, статусы задач
- ✓ Финальное тестирование и polish UI
- ✓ Деплой и демо-презентация

10. Почему этот проект важен

Реальная боль

Проблема существует прямо сейчас среди студентов университета. Это не придуманная задача — это реальный барьер при трудоустройстве.

Осязаемый результат

Проект можно показать и пощупать: открыть в браузере, написать строку кода, нажать Run и увидеть результат прямо на презентации.

Современный стек

AI + FastAPI + React — это индустриальный уровень. Такой стек выглядит серьёзно даже для студенческого проекта.

Масштабируемо сть

Платформа легко расширяется: новые языки программирования, больше проектов, интеграция с LMS университета, рейтинги и соревнования.